Grenoble INP - Ensimag, UGA
Projet GL 2024-2025

Documentation de validation

Groupe 6 Equipe 34

BORNARD Mael, FOKOU DJONTU Manuella, GAUTIER Mattéo,
JEULIN Matteo, MELLOT Arthur

Table des matières :

Table des matières : **2**
1. Descriptif des Tests 3
2. Organisation des tests **3**
3. Objectifs des tests **5**
4. Scripts de tests **5**
5. Comment passer tous les tests **6**
6. Gestion des risques et gestion des rendus **7**
a. Gestion des risques 7
b. Gestion des rendus 7
c. Gestion de mise en production/ de merge 7
7. Résultats de couverture générés avec Jacoco **7**
a. Couverture Globale 8
b. Détail par dossier 8
8. Méthodes de validation autres que les tests **11**

1. Descriptif des Tests

Les tests sont situés dans un dossier principal nommé **test(src/test)**.

Ce dossier est organisé en trois sous-dossiers :

1. **deca** : Contient les cas de tests deca spécifiques à chaque partie de notre compilateur (analyse syntaxique, contexte, génération de code, décompilation).
2. **java** : Non modifié dans notre projet
3. **script** : Contient des scripts permettant d'exécuter les tests automatiquement.

2. Organisation des tests

Pour chaque étape du compilateur (analyse syntaxique, analyse contextuelle et génération de code), nous avons conçu des tests valides et des tests invalides. Ces tests sont organisés selon deux critères :

1. **Validité** (valide ou invalide).
2. **Fonctionnalité spécifique** (par exemple, print, variables, opérations, classes, etc.).

Tests valides

- **Objectif** : Vérifier que les programmes passent sans erreur et produisent un résultat conforme aux attentes.
- **Résultat attendu** : Documenté en début de fichier dans les commentaires, encadré par les balises `[Begin Result]` et `[End Result]`.

Tests invalides

- **Objectif** : Vérifier que les erreurs appropriées sont détectées.
- **Types d'erreurs** : Lexicales, syntaxiques, contextuelles ou liées à l'exécution.
- **Résultat attendu** : L'erreur produite doit correspondre à celle attendue, décrite en début de fichier dans les commentaires, encadrée par `[Begin Result]` et `[End Result]`.

Les tests sont organisés pour correspondre directement aux étapes du compilateur :

- **deca/syntax** pour l'analyse syntaxique.

  Tests lexicalement invalides : Vérifient le bon fonctionnement du lexer.

Tests syntaxiquement invalides : Vérifient le parseur. Ces fichiers incluent un suffixe `_<numéro_de_ligne>` dans leur nom, où `<numéro_de_ligne>` correspond à la ligne provoquant l'erreur.

Tests valides : Vérifient les programmes qui passent à la fois l'analyse lexicale et syntaxique (validation de l'étape A).

- **deca/context** pour l'analyse contextuelle.
- **deca/codegen** pour la génération de code.

  Tests valides : Vérifient que la sortie produite correspond aux résultats attendus.

  Tests invalides : Les programmes invalides doivent générer une erreur à l'exécution, sauf si l'option `-n` est spécifiée lors de la compilation. Avec `-n`, aucune erreur n'est levée par le compilateur, mais des erreurs peuvent survenir à l'exécution en cas d'instructions invalides.

  Programmes interactifs : Pour les programmes nécessitant une entrée utilisateur (`readInt`, `readFloat`), les données à entrer sont documentées dans les commentaires en début de fichier, entre les balises `[Begin Input]` et `[End Input]`.

- **deca/decompile** pour la décompilation(option -p du décompilateur) .
- **game** : contient des programmes deca plus complexes. Il s'agit de jeux auquel l'utilisateur peut jouer dans le terminal

Les tests Java n'ont pas été modifiés, et leur validation reste celle par défaut.
Les scripts nécessaires pour exécuter ces tests sont centralisés dans le dossier **script**.

3. Objectifs des tests

L'objectif des tests est de tester l'ensemble des chemins possibles que peut prendre le compilateur et de provoquer toutes les erreurs possibles. Pour cela nous nous appuyons sur les règles de la grammaire.

- **Objectifs principaux :**
  - Garantir la conformité au langage spécifié.
  - Identifier et corriger les bogues à chaque passe.
  - Assurer une couverture maximale du code (via Jacoco).
- **Résultats atteints :** voir la partie couverture des tests.

4. Scripts de tests

Nous avons créé plusieurs scripts qui testent chacun une partie du compilateur sur des programmes valides et invalides.
Les scripts situés dans le dossier **src/test/script** et **src/test/script/launcher** permettent d'exécuter facilement les tests. Pour lancer ces tests, se référer à la section suivante.
Pour chaque fichier tester chacun des scripts affiche le succès ou l'échec du test de manière claire et concise. Si le test échoue, le résultat attendu et celui obtenu sont affichés pour faciliter le débogage. A la fin de chaque script, le pourcentage de tests validés est affiché.

- **advanced-lex.sh** : ce script a pour but de tester le fonctionnement du lexer (partie A). Il utilise le script test_lex sur les fichiers du dossier **src/test/deca/syntax**. Pour les tests valides, on vérifie qu'il n'y ait pas d'erreur et pour les tests invalides, on vérifie que l'erreur est levé à la bonne ligne
- **advanced-synt.sh** : ce script a pour but de tester le fonctionnement du parser (partie A). Il utilise le script test_synt sur les fichiers du dossier **src/test/deca/syntax**. Son fonctionnement est identique au script précédent.
- **advanced-context.sh** : ce script à pour but de tester la vérification contextuelle (partie B). Il utilise le script test_context sur les fichiers du dossier **src/test/deca/context**. Pour les tests valides, on vérifie qu'il n'y ait pas d'erreur et pour les tests invalides, on vérifie que l'erreur levée est exactement la même que celle attendue (ligne, caractère, message).
- **advanced-codgen.sh** : ce script à pour but de tester la génération de code assembleur (partie C). Il compile les fichiers du dossier **src/test/deca/codegen** avec decac puis exécute avec ima les fichier assembleurs généré. Pour les tests valides, on vérifie qu'il n'y ait pas d'erreur à la compilation et que la sortie standard soit conforme à celle attendue. Pour les tests invalides, on vérifie qu'il n'y ait pas d'erreur de compilation et que l'erreur levée à l'exécution est conforme à celle attendue.
- **advanced-decac.sh** : ce script à pour but de tester les différentes options du compilateur decac. Selon les options, le script vérifie la bonne compilation, la conformité de la sortie standard, l'absence d'erreur à l'exécution (-n), les registres utilisés dans le fichiers assembleur généré (-r) ...
- **decompile.sh** : ce script à pour but de tester à la fois l'option -p du compilateur et de vérifier que la création de l'arbre (étape 3) est correcte. Le principe est le suivant : décompiler (decac -p) le fichier .deca et sauvegarder le résultat dans un nouveau fichier .deca, décompiler ce nouveau fichier, vérifier que le programme produit est identique au premier programme décompilé.
- **test_all** : ce script à pour but de tester toutes les parties du projet. Il exécute un à un les scripts ci-dessus. A la fin de son exécution, nous obtenons le pourcentage de tests passés ainsi que le nombre total d'erreurs et le nombre total de tests exécutés.

5. Comment passer tous les tests

- Afin de faire d'exécuter l'ensemble des tests présent sur ce projet (tests fournis + test implémentés), il suffit de lancer la commande : `mvn verify`
- Pour exécuter l'ensemble des test et calculer la couverture avec Jacoco il faut lancer la commande suivant à la racine du projet :
  `mvn -Djacoco.skip=false`
  Puis pour visualiser le rapport produit :
  `firefox target/site/jacoco/index.html`

- Pour lancer l'ensemble de tests que nous avons nous même implémentés il faut compiler les tests (`mvn test-compile`) et le code source (`mvn compile`) puis lancer le script : `src/test/script/launchers/test_all`
- Pour lancer les tests spécifique à chaque partie, compiler puis lancer
  - `src/test/script/advanced-lex.sh`
  - `src/test/script/advanced-synt.sh`
  - `src/test/script/advanced-context.sh`
  - `src/test/script/advanced-codgen.sh`
  - `src/test/script/advanced-decac.sh`
  - `src/test/script/decompile.sh`

6. Gestion des risques et gestion des rendus

a. Gestion des risques

- Risque : compréhension erronée du sujet par un membre de l'équipe.
  Action : idéalement tests et implémentation par 2 personnes différentes.
  Concrètement, pair programming lors des débuggages complexes, communication constante.
- Risque : ajout d'un bug non détecté.
  Action : création d'un jeu de tests couvrant toutes les possibilités d'erreurs pour le détecter le plus rapidement possible.

- Risque : push une version du code ne compilant pas.
  Action : compiler avant de push ou ajouter une pipeline à GitLab permettant de lancer automatiquement des tests au moment d'un commit.

- Risque : oublier les deadlines

Action : Intégration au Trello des dates de suivis, de rendus de code et de documentation, avec le contenu à rendre et les consignes. Envoi de rappel un jour avant et popup dans le tableau de bord afin d'informer tous les membres du groupes.

b. Gestion des rendus

- Dépôt Git structuré avec des branches pour les fonctionnalités et corrections. Une fois une version du code validée pour le rendu et présentée sur la branche master, il est interdit de rajouter du code au dépôt gitlab sur la branche master avant l'heure du rendu. Cela garantit qu'aucun code puisse, par erreur, "casser" le rendu.
- Finir le développement des fonctionnalités 1 ou 2 jours avant le rendu afin d'avoir le temps de tester le compilateur. En cas de détection de bugs, cela nous laisse le temps de corriger puis de tester à nouveau. De plus, cela évite de travailler sous la pression de la deadline approchante qui souvent est plus néfaste que bénéfique pour le produit rendu.
- Une idée que nous avons eu à la fin du projet et qui aurait pu être utile est l'utilisation de tags pour marquer les versions stables.

c. Gestion de mise en production/de merge

- S'assurer que tous les membres de l'équipe ont bien push ce qu''ils devaient en vu du rendu.
- Récupérer la dernière version du code présent sur le dépôt distant sur chaque branche concernée (`git pull`).
- Merger l'ensemble des features fonctionnelles sur la branche master/dev.
- Bien se positionner sur la branche master/dev, faire un `mvn clean` puis recompiler avec `mvn test-compile` et `mvn compile`.
- Lancer `test_all` et vérifier que tous les tests passent. Cela garantit le bon fonctionnement du compilateur.

7. Résultats de couverture générés avec Jacoco

a. Couverture Globale

Dans l'ensemble du projet, 82% des instructions sont couvertes par des tests, ce qui représente 71% des branches.

Deca Compiler

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
fr.ensimag.deca.syntax		74%		58%	463	667	484	2,024	255	371	4	48
fr.ensimag.deca.tree		94%		88%	78	704	101	1,898	27	474	0	85
fr.ensimag.deca		71%		61%	50	115	78	319	8	54	2	5
fr.ensimag.ima.pseudocode		81%		81%	25	88	35	189	21	77	1	27
fr.ensimag.deca.context		89%		80%	22	138	28	234	16	118	0	21
fr.ensimag.ima.pseudocode.instructions		74%		n/a	18	64	32	116	18	64	14	54
fr.ensimag.deca.tools		94%		100%	1	16	3	39	1	13	0	3
Total	3,837 of 22,186	82%	334 of 1,182	71%	657	1,792	761	4,819	346	1,171	21	243

Figure 1 : Couverture globale des tests avec Jacoco

b. Détail par dossier

- **Dossier Syntax**

Dans le dossier syntaxe, 74% des instructions sont couvertes. La majorité des instructions non couvertes sont dans le fichier DecaParser généré par Antlr.

- **Dossier Tree**

Ce dossier contient les fichiers formant l'arborescence de l'arbre construit à l'étape A et décoré à l'étape B. Dans ce dossier, 94% des instructions sont couvertes. Les parties du code non couvertes sont soit des fonctions non utilisées dans le projet ou des levées d'exceptions internes (par exemple InternalError) qui dans le fonctionnement normal du compilateur ne doivent jamais être levées.

fr.ensimag.deca.tree

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Identifier		73%		75%	8	36	18	78	1	22	0	1
AbstractDeclClass		81%		80%	4	23	4	74	0	13	0	1
Tree		78%		81%	6	30	13	64	4	22	0	1
AbstractExpr		79%		81%	8	23	6	38	5	15	0	1
DeclField		86%		87%	2	11	11	51	1	7	0	1
AbstractDeclField		88%		87%	1	8	3	24	0	4	0	1
ConvFloat		57%		75%	3	6	3	10	2	4	0	1
AbstractDeclMethod		90%		80%	4	14	1	27	0	4	0	1
IfThenElse		92%		75%	2	10	8	66	1	8	0	1
DeclVar		93%		100%	0	8	4	51	0	6	0	1
FieldAccess		95%		85%	3	20	2	51	0	10	0	1
DeclParam		91%		100%	1	8	1	29	1	7	0	1
Assign		90%		66%	2	8	2	21	0	5	0	1
LocationException		86%		50%	3	7	2	14	0	4	0	1
Cast		96%		87%	4	25	1	50	0	9	0	1
NoOperation		53%		n/a	3	6	3	7	3	6	0	1
Initialization		88%		n/a	1	9	3	20	1	9	0	1
DeclClass		98%		100%	1	18	1	61	1	9	0	1
AbstractOpArith		96%		82%	5	17	1	25	0	3	0	1
NoInitialization		93%		87%	1	12	1	18	0	8	0	1
AbstractLValue		84%		75%	1	5	1	8	0	3	0	1
Program		96%		n/a	0	9	3	42	0	9	0	1
TreeList		96%		100%	1	14	1	23	1	11	0	1
Location		92%		50%	2	8	2	14	0	6	0	1
New		98%		100%	1	9	1	29	1	8	0	1
AbstractPrint		98%		100%	1	15	1	34	1	8	0	1
StringLiteral		95%		n/a	1	9	1	15	1	9	0	1
BooleanLiteral		95%		100%	1	9	1	13	1	8	0	1
FloatLiteral		97%		50%	2	12	0	19	0	10	0	1
AbstractBinaryExpr		99%		89%	3	27	0	73	0	13	0	1
ReadFloat		97%		n/a	1	8	1	16	1	8	0	1
ReadInt		97%		n/a	1	7	1	14	1	7	0	1
MethodCall		100%		100%	0	12	0	66	0	8	0	1
ClassicDeclMethod		100%		100%	0	10	0	70	0	7	0	1
InstanceOf		100%		100%	0	8	0	53	0	6	0	1
ListDeclClass		100%		100%	0	16	0	47	0	9	0	1
While		100%		100%	0	9	0	44	0	8	0	1

Figure 2 : Couverture des tests dans le dossier tree (extrait, les fichiers non affichés sont à 100% de couverture)

- **Dossier Deca**

Dans ce dossier, 71% des instructions sont couvertes. La première partie du code non couvert concerne le rattrapage des exceptions internes au compilateur dans le fichier DecacCompiler.java. La seconde partie concerne la levée d'exception (CLIException) lors du parsing des arguments de la ligne de commande dans le fichier CompilerOptions.java. Cette partie du code aurait pu être couverte par les tests mais cela ne nous semblait pas prioritaire de tester ces erreurs de manière automatique, nous les avons testés à la main lors de l'implémentation.

fr.ensimag.deca

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
DecacCompiler		77%		57%	17	55	34	181	3	34	0	1
CompilerOptions		59%		61%	25	45	24	92	2	14	0	1
DecacMain		69%		72%	6	13	16	42	1	4	0	1
CLIException		0%		n/a	1	1	2	2	1	1	1	1
DecacFatalError		0%		n/a	1	1	2	2	1	1	1	1
Total	410 of 1,418	71%	46 of 119	61%	50	115	78	319	8	54	2	5

Figure 3 : Couverture des tests dans le dossier deca

- **Dossier context**

89% des instructions sont couvertes par les tests dans ce dossier

fr.ensimag.deca.context

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
EnvironmentType		96%		n/a	0	3	2	31	0	3	0	1
ClassDefinition		98%		75%	1	17	0	30	0	15	0	1
Definition		74%		100%	5	12	6	20	5	11	0	1
ClassType		90%		87%	2	12	3	19	1	8	0	1
MethodDefinition		59%		0%	5	12	6	17	4	11	0	1
Signature		93%		75%	2	9	2	17	0	5	0	1
EnvironmentExp		81%		75%	2	8	3	16	1	4	0	1
Type		100%		100%	0	16	0	16	0	14	0	1
FieldDefinition		93%		n/a	1	8	1	12	1	8	0	1
FloatType		100%		n/a	0	5	0	8	0	5	0	1
IntType		100%		100%	0	7	0	7	0	5	0	1
ParamDefinition		81%		n/a	1	4	1	5	1	4	0	1
ExpDefinition		100%		n/a	0	3	0	5	0	3	0	1
NullType		100%		n/a	0	4	0	5	0	4	0	1
EnvironmentExp.DoubleDefException		57%		n/a	1	2	2	4	1	2	0	1
StringType		77%		n/a	1	3	1	4	1	3	0	1
TypeDefinition		77%		n/a	1	3	1	4	1	3	0	1
BooleanType		100%		n/a	0	3	0	4	0	3	0	1
VariableDefinition		100%		n/a	0	3	0	4	0	3	0	1
VoidType		100%		n/a	0	3	0	4	0	3	0	1
ContextualError		100%		n/a	0	1	0	2	0	1	0	1
Total	82 of 813	89%	8 of 40	80%	22	138	28	234	16	118	0	21

Figure 4 : Couverture des tests dans le dossier context

8. Méthodes de validation autres que les tests

Nous avons fait le choix de ne pas implémenter de tests unitaires en java. En effet nos fichiers de tests en deca étaient assez précis et couvrants pour détecter tout problème dans le code source.